

ESO-207: Theoretical Assignment-2

Aviral Gupta
230246

Indian Institute of Technology, Kanpur

Question 1: The Archive Sorting Ritual

In the Great Archive of Chronos, priceless scrolls containing the kingdom's history are stored on long shelves. The librarians must keep these scrolls in increasing chronological order (from year 1 to year n), but after festivals, the scrolls are often returned in jumbled order.

The Archive's enchanted shelves perform a special sorting ritual. If a segment of scrolls is already in strictly increasing order, the ritual halts there - no energy is spent sorting it further. Otherwise, the segment is divided into two halves, each half is sorted recursively, and finally the two halves are merged together. The ritual is described below:

Algorithm 1: ArchiveSort($A[l..r]$)

```
1 if  $A[l..r]$  is in strictly increasing order then
2   return;
3 if  $l < r$  then
4    $mid \leftarrow \lfloor (l+r)/2 \rfloor$ ;
5   ArchiveSort( $A[l..mid]$ );
6   ArchiveSort( $A[mid+1..r]$ );
7   Merge( $A[l..mid]$ ,  $A[mid+1..r]$ );
```

Each invocation of **ArchiveSort** consumes one unit of magical energy. The Council of Mages has decreed that exactly k units of energy must be consumed as:

- If fewer than k units are used, the ritual is incomplete, and the scrolls remain unstable.
- If more than k units are used, the excess energy may backfire, damaging the shelves.

Thus, the Head Librarian must carefully arrange the scrolls before sorting so that the ritual consumes exactly k units. Formally:

- You are given two integers n and k .
- Construct a permutation of $1, 2, \dots, n$ such that the above ritual is invoked exactly k times.

Question (a)

Design an algorithm that, given n and k , constructs a permutation of $1, 2, \dots, n$ such that ArchiveSort is invoked exactly k times. Provide pseudocode and explain the construction clearly.

Solution:

Algorithm Idea

The goal is to construct a permutation of length n that forces the recursive ArchiveSort procedure to consume exactly k calls.

- Each call to ArchiveSort corresponds to one node in the recursion tree.

- The base case ($k = 1$) occurs when the segment is already sorted in strictly increasing order. Hence, we simply output the natural order $[1, 2, \dots, n]$.
- If $k > 1$, then we must force the recursion to continue. We split the array into two halves of sizes

$$n_1 = \lfloor n/2 \rfloor, \quad n_2 = n - n_1.$$

The current call consumes 1 unit of energy, and the remaining $k - 1$ must be distributed between the two halves:

$$k = 1 + k_1 + k_2.$$

- To guarantee that the entire segment is not sorted at the root, we assign the *larger* numbers to the left half and the *smaller* numbers to the right half. This ensures that the array as a whole is not increasing, forcing a split.
- Finally, we recursively build permutations for both halves and concatenate them.

Pseudocode

Algorithm 2: BUILDPERMUTATION($n, k, ans, start, base$)

Input: Size n , required calls k , answer array ans , offset $start$, base value $base$

Output: Fills $ans[start..start + n - 1]$ with the desired permutation

```

1 if  $k < 1$  or  $k > 2n - 1$  or  $k$  even then
2   return FAIL // No valid permutation
3 if  $k = 1$  then
4   for  $j \leftarrow 0$  to  $n - 1$  do
5      $ans[start + j] \leftarrow base + j$ 
6   return
7  $n_1 \leftarrow \lfloor n/2 \rfloor$ 
8  $n_2 \leftarrow n - n_1$ 
9  $k_1 \leftarrow \min(2n_1 - 1, k - 2)$ 
10  $k_2 \leftarrow (k - 1) - k_1$ 
11 BUILDPERMUTATION( $n_1, k_1, ans, start, base + n_2$ )
12 BUILDPERMUTATION( $n_2, k_2, ans, start + n_1, base$ )

```

Question (b)

Prove the correctness of your construction. Why does your permutation guarantee exactly k recursive calls?

Solution:

Proof of Correctness-

Base case ($n = 1$)

The only permutation of a single element is trivially sorted, so ArchiveSort invoked on that segment makes exactly one call. The only allowed value of k is $k = 1$ (since $1 \leq k \leq 2 \cdot 1 - 1 = 1$ and k must be odd). Our construction returns the single-element sorted block $[base]$, so the claim holds.

Inductive hypothesis

Assume the claim holds for all sizes smaller than n (for every valid odd k in the corresponding ranges).

Inductive step

Consider size n and an odd k with $1 \leq k \leq 2n - 1$. There are two cases.

Case 1: $k = 1$. By construction we return the sorted block $[base, \dots, base + n - 1]$. Since the whole segment is strictly increasing, ArchiveSort stops at the root and therefore makes exactly one call. The claim holds.

Case 2: $k > 1$. Then $k \geq 3$ and is odd. Split n into

$$n_1 = \lfloor n/2 \rfloor, \quad n_2 = n - n_1.$$

We choose

$$k_1 = \min(2n_1 - 1, k - 2), \quad k_2 = (k - 1) - k_1.$$

We must check these choices are valid and then show the construction yields exactly k calls.

(i) Parity and bounds of k_1, k_2

- $2n_1 - 1$ is odd, and $k - 2$ is odd because k is odd. Hence k_1 is the minimum of two odd positive integers, so k_1 is odd.
- $k - 1$ is even, so $k_2 = (k - 1) - k_1$ is even minus odd, therefore odd.
- Since $k \leq 2n - 1$ we have $k - 2 \leq 2n - 3$. If $k_1 = k - 2$ then $1 \leq k_1 \leq 2n_1 - 1$ because $k - 2 \geq 1$ (as $k \geq 3$) and by the min definition $k_1 \leq 2n_1 - 1$. If $k_1 = 2n_1 - 1$ then obviously $1 \leq k_1 \leq 2n_1 - 1$. Thus $1 \leq k_1 \leq 2n_1 - 1$. Finally

$$k_2 = (k - 1) - k_1 \leq (k - 1) - 1 = k - 2 \leq 2n - 3 \leq 2n_2 - 1,$$

where the last inequality follows from $n_2 \geq 1$ and simple algebra. Therefore $1 \leq k_2 \leq 2n_2 - 1$. Hence k_1, k_2 are valid odd integers in the allowed ranges for sizes n_1, n_2 respectively.

(ii) Recursive construction and permutation property

By the inductive hypothesis we construct

$$L = \text{BUILDPERMUTATION}(n_1, k_1, base + n_2) \quad \text{and} \quad R = \text{BUILDPERMUTATION}(n_2, k_2, base),$$

where L is a permutation of $\{base + n_2, \dots, base + n - 1\}$ and R is a permutation of $\{base, \dots, base + n_2 - 1\}$. Because the two value-ranges are disjoint and their union equals $\{base, \dots, base + n - 1\}$, the concatenation $L \parallel R$ is a permutation of the whole set $\{base, \dots, base + n - 1\}$.

Why ArchiveSort makes exactly k calls on the constructed array

- The root call always occurs (we always run ArchiveSort on the whole segment), so count at least 1.
- The array $L \parallel R$ is *not* strictly increasing across the border between the left and right blocks because every element in L is larger than every element in R . Thus the root segment is not sorted and ArchiveSort must split the segment into the two halves and recurse on both halves.
- By the inductive hypothesis, ArchiveSort applied to the left half invokes exactly k_1 calls (because the left half equals the permutation constructed for parameters (n_1, k_1)) and applied to the right half invokes exactly k_2 calls. These two recursive calls together with the root call give a total of

$$1 + k_1 + k_2 = k.$$

Hence the whole process uses exactly k invocations.

This completes the inductive step. By induction the construction is correct for every $n \geq 1$ and every odd k with $1 \leq k \leq 2n - 1$: the algorithm produces a permutation of $\{base, \dots, base + n - 1\}$ and ArchiveSort on that permutation makes exactly k calls.

Question (c)

Analyze the time complexity of your construction algorithm.

Solution:

Time Complexity:

- **Leaf work:** Whenever $k = 1$, the algorithm directly writes a sorted block of size n' . Across all leaves, these blocks form a partition of the n elements, so the total writing cost is $\Theta(n)$.
- **Number of calls:** From part (b), the recursion tree is built to have exactly k nodes (each node corresponds to one call of ARCHIVESORT).
- **Internal work:** Each non-leaf call performs only constant work (splitting n into n_1, n_2 , computing k_1, k_2), independent of n . Over k calls, this adds up to $O(k)$.
- **Total time:** Adding both contributions,

$$T(n, k) = \Theta(n) + \Theta(k) = \Theta(n + k).$$

Space Complexity:

The output array requires $\Theta(n)$ memory. The recursion depth is $\Theta(\log n)$ since n is split roughly in half at each step.

Question 2: Royal Guard Deployment

In the kingdom of Algoria, commanders are organized in a binary search tree (BST) based on their rank. Each commander has:

- a **key** representing their rank,
- a **strength value** representing their combat power.

The king wants to select a group of commanders as royal guards. There is one rule for selection:

- **Parent-Child Rule:** If a commander is selected as a royal guard, none of their **children in the BST** can be selected. This is because each parent and child commander are friends and will spend time talking instead of guarding.

Parent-child pairs are not selected together, respecting their friendship rule.

You have to select a subset of commanders maximizing the total strength of the royal guard while respecting the Parent-Child Rule.

Question (a)

Design an algorithm in $\Theta(n)$ time and write a clear pseudocode.

Solution:

Our goal is to compute both:

1. the maximum possible strength, and
2. the subset of commanders (by key) that achieves this maximum.

Algorithmic Idea-

For each node u , we maintain two values:

- $\text{Include}[u]$: maximum strength if u is included,
- $\text{Exclude}[u]$: maximum strength if u is excluded.

Recurrence relations:

$$\text{Include}[u] = \text{strength}[u] + \sum_{\text{child } v} \text{Exclude}[v],$$

$$\text{Exclude}[u] = \sum_{\text{child } v} \max(\text{Include}[v], \text{Exclude}[v]).$$

The maximum total strength is:

$$\max(\text{Include}[\text{root}], \text{Exclude}[\text{root}]).$$

Reconstruction of subset:

- Start at the root.
- If $\text{Include}[u] > \text{Exclude}[u]$, then u is chosen, and its children are excluded.
- Otherwise, u is excluded, and for each child v we decide recursively based on whether $\text{Include}[v] > \text{Exclude}[v]$.

Pseudocode:**Algorithm 3:** RoyalGuardSelection**Input:** Root of the BST, where each node has key, strength, left, and right**Output:** Maximum total strength and the set of selected commander keys

```

1 Function Post-Order Traversal(node):
2   if node = NULL then
3     return (0, 0)
4   (incLeft, excLeft) ← Post-Order Traversal(node.left);
5   (incRight, excRight) ← Post-Order Traversal(node.right);
6   Include[node] ← node.strength + excLeft + excRight;
7   Exclude[node] ← max(incLeft, excLeft) + max(incRight, excRight);
8   return (Include[node], Exclude[node]);

9 Function BuildSet(node, take, result):
10  if node = NULL then
11    return
12  if take = TRUE then
13    result.add(node.key);
14    BuildSet (node.left, FALSE, result);
15    BuildSet (node.right, FALSE, result);
16  else
17    BuildSet (node.left, (Include[node.left] > Exclude[node.left]), result);
18    BuildSet (node.right, (Include[node.right] > Exclude[node.right]), result);

19 (IncludeRoot, ExcludeRoot) ← Post-Order Traversal(root);
20 BuildSet (root, (IncludeRoot > ExcludeRoot), SelectedSet);
21 return (max(IncludeRoot, ExcludeRoot), SelectedSet);

```

Question (b)

Give time complexity analysis of your approach for part (a).

Solution:

We analyze the algorithm from part (a), which has two main phases:

1. Postorder Traversal

- We perform a postorder traversal of the BST.
- At each node u , we compute two values:

$$\text{Include}[u], \quad \text{Exclude}[u]$$

using constant-time arithmetic and comparison operations.

- The recurrence depends only on the values of the children, so once the recursive calls return, the result for u can be computed in $\Theta(1)$ time.
- Since there are n nodes, the total work in this phase is:

$$\Theta(1) \times n = \Theta(n).$$

2. Reconstruction of the Subset-

- Starting from the root, we decide whether to include or exclude each node.
- Each node is visited once during this reconstruction, and at each node we perform at most a constant number of operations.
- Therefore, the reconstruction also takes:

$$\Theta(n).$$

Overall Complexity: Combining the two phases:

$$\Theta(n) + \Theta(n) = \Theta(n),$$

Space Complexity:

- We require $\Theta(n)$ space to store the two values (Include and Exclude) for each node.
- The recursion stack of DFS has depth $\Theta(h)$, where h is the height of the tree. In the worst case (a skewed BST), $h = \Theta(n)$, while in a balanced tree, $h = \Theta(\log n)$.

Thus, the total space complexity is $\Theta(n)$ in the worst case.

Question 3: As Pretty As It Gets!

IIT Kanpur is setting up a brand-new row of halls of residence along the road to the Nankari gate. Each plot i has an engineering limit m_i , which represents the tallest possible hall that can be built there.

But here's the catch: the Hall Executive Committee insists that the row of hostels must look **pretty in the skyline photos** (especially for the next IITK brochure). To ensure this:

- The height a_i of the hall at plot i must satisfy $1 \leq a_i \leq m_i$.
- No hall should look like a “dip” between two taller halls. Formally, there must not exist indices $j < i < k$ such that $a_j > a_i$ and $a_k > a_i$.

The Institute wants to maximize the **total number of floors across all halls**, i.e., $\sum a_i$, while keeping the skyline picturesque.

Your task is to figure out the optimal distribution of heights $a_1, a_2, \dots, a_n$.

Question (a)

Describe a brute-force algorithm to assign hall heights while satisfying the rules. Explain how it works and analyze its time complexity.

Solution:

Brute Force Algorithm-

The brute-force algorithm explores every possible way of assigning hall heights within the given limits.

1. For each plot i , the height a_i can take any value between 1 and m_i . Hence, the algorithm enumerates all possible sequences $(a_1, a_2, \dots, a_n)$, where $1 \leq a_i \leq m_i$. This covers every possible skyline configuration.
2. For each candidate sequence, we check if it satisfies the condition. To do this, we scan from left to right:
 - First, the heights should be non-decreasing (buildings are getting taller or staying the same).
 - At some peak, the heights stop increasing.
 - After the peak, the heights should be non-increasing (buildings are getting shorter or staying the same).

If the property is not satisfied, we reject it and continue further.

3. If the sequence is valid, we calculate the total number of floors by summing up all chosen heights $\sum_{i=1}^n a_i$.
4. The algorithm keeps track of the **maximum total sum** and remembers the corresponding skyline.

Time Complexity Analysis-

- Number of arrangements: $\prod_{i=1}^n m_i$.
- Checking if it is unimodal: $O(n)$ per arrangement.
- Computing sum: $O(n)$ per arrangement.

Thus, the total complexity is:

$$O\left(n \cdot \prod_{i=1}^n m_i\right),$$

which is exponential in n (i.e., $O(M^n \cdot n)$ if $\text{Max } m_i = M$).

Pseudocode:**Algorithm 4:** BruteForcePrettySkyline

Input: Array of limits $m[1 \dots n]$
Output: Optimal heights $a[1 \dots n]$ maximizing total floors

```

1 maxSum  $\leftarrow -\infty$ ;
2 bestAssignment  $\leftarrow \text{null}$ ;
3 foreach assignment  $a[1 \dots n]$  where  $1 \leq a[i] \leq m[i]$  do
4   if isUnimodal( $a$ ) then
5     total  $\leftarrow \sum_{i=1}^n a[i]$ ;
6     if total > maxSum then
7       maxSum  $\leftarrow$  total;
8       bestAssignment  $\leftarrow a$ ;
9 return bestAssignment;
10 Function isUnimodal( $a[1 \dots n]$ ):
11    $i \leftarrow 1$ ;
12   while  $i < n$  and  $a[i] \leq a[i + 1]$  do
13      $i \leftarrow i + 1$ ;
14   while  $i < n$  and  $a[i] \geq a[i + 1]$  do
15      $i \leftarrow i + 1$ ;
16   return ( $i = n$ );

```

Question (b)

Propose an efficient algorithm that computes the hall heights in $\Theta(n)$ time. Provide pseudocode for your method.
Hint : Try using a *stack – based approach* to get the final height in two passes.

Solution:**Algorithmic Idea-**

The skyline must therefore have a single *peak*, increasing (weakly) to the peak and decreasing (weakly) thereafter. The challenge is to find the best peak efficiently.

The algorithm uses a *stack-based approach* to compute two arrays:

- $leftmx[i]$: maximum sum of heights achievable for the prefix ending at $i - 1$, if plot $i - 1$ is the tallest element in that prefix.
- $rightmx[i]$: maximum sum of heights achievable for the suffix starting at i , if plot i is the tallest element in that suffix.

Both arrays are computed in linear time using a monotone stack, ensuring that we can efficiently extend contributions from smaller elements to larger ranges.

Once $leftmx$ and $rightmx$ are known, the algorithm tries each index i as a potential peak. For each candidate peak:

$$\text{total}(i) = leftmx[i + 1] + rightmx[i] - v[i],$$

where we subtract $v[i]$ because it gets counted twice (once in left, once in right). The best i becomes the chosen peak. Finally, the skyline is reconstructed by starting from the peak and propagating heights outward:

- On the left, each hall is limited by both its engineering bound and the height of its right neighbor.
- On the right, each hall is limited by its engineering bound and the height of its left neighbor.

This guarantees that the arrangement is satisfying the conditions of the question.

Pseudocode:**Algorithm 5: StackBasedSkyline (pseudocode form)**

Input: Number of plots n , limits $v[0 \dots n-1]$
Output: Optimal skyline $res[0 \dots n-1]$

- 1 **Phase 1: Compute left contributions;**
- 2 Initialize array $leftmx[0 \dots n] \leftarrow 0$;
- 3 Initialize empty stack S ;
- 4 **for each index i from 0 to $n-1$ do**
- 5 Remove from S all indices whose $v[\cdot]$ value is larger than $v[i]$;
- 6 Let $a \leftarrow (S \text{ not empty})?(S.top + 1) : 0$;
- 7 Set $leftmx[i+1] \leftarrow leftmx[a] + v[i] \cdot (i - a + 1)$;
- 8 Push i onto S ;
- 9 **Phase 2: Compute right contributions;**
- 10 Initialize array $rightmx[0 \dots n] \leftarrow 0$;
- 11 Clear stack S ;
- 12 **for each index i from $n-1$ down to 0 do**
- 13 Remove from S all indices whose $v[\cdot]$ value is larger than $v[i]$;
- 14 Let $a \leftarrow (S \text{ not empty})?(S.top) : n$;
- 15 Set $rightmx[i] \leftarrow rightmx[a] + v[i] \cdot (a - i)$;
- 16 Push i onto S ;
- 17 **Phase 3: Choose the best peak;**
- 18 Initialize $ans \leftarrow -\infty, peak \leftarrow -1$;
- 19 **for each index i from 0 to $n-1$ do**
- 20 Compute $total \leftarrow leftmx[i+1] + rightmx[i] - v[i]$;
- 21 If $total$ improves ans , update ans and record $peak \leftarrow i$;
- 22 **Phase 4: Reconstruct skyline;**
- 23 Set $res[peak] \leftarrow v[peak]$;
- 24 Fill left side by moving from $peak-1$ down to 0: $res[i] \leftarrow \min(v[i], res[i+1])$;
- 25 Fill right side by moving from $peak+1$ up to $n-1$: $res[i] \leftarrow \min(v[i], res[i-1])$;
- 26 **return res ;**

Question (c)

Analyze the time complexity of your optimized algorithm for part (b).

Solution:

The algorithm consists of four phases:

- **Left pass:** Each index is pushed onto the stack once and popped at most once. Thus, the total cost of the left pass is $\Theta(n)$.
- **Right pass:** Symmetrically, each index is pushed and popped at most once. The total cost is again $\Theta(n)$.
- **Choosing the peak:** We iterate once through all n positions to compute the best total sum. This costs $\Theta(n)$.
- **Reconstruction:** From the chosen peak, we walk leftwards once and rightwards once. This also takes $\Theta(n)$ time.

Adding up all phases:

$$T(n) = \Theta(n) + \Theta(n) + \Theta(n) + \Theta(n) = \Theta(n).$$

Hence, the optimized stack-based skyline algorithm runs in overall linear time.